

同济大学计算机系

数字逻辑课程实验报告



学 号 2351520

姓 名 毛星博

专 业 计算机科学与技术

授课老师 张冬冬

6.6_1 同步复位 D 触发器

一、实验内容

b) D 触发器

D 触发器是在 SR 触发器基础上构建的。即在一个 SR 触发器上增加一个非门。图 6.6.2 给出了一个 D 触发器的逻辑图及其功能表。该触发器只有一个数据输入端 D，经反相器反相后，变成互补数据输入，送到 SR 触发器，从而避免了 SR 触发器存在的不稳态问题。当数据输入 D=1，且在时钟脉冲上升沿，触发器置位（1 状态）；当数据输入 D=0，且在时钟脉冲上升沿，触发器复位（0 状态）。

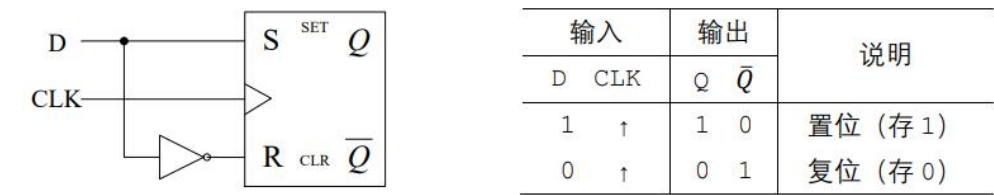


图 6.6.2 D 触发器逻辑图及其功能表

所谓同步复位，指的是同步复位信号就是指复位信号只在所需时钟边沿到来时才有效。

二、硬件逻辑图：无

三、模块建模

（该部分要求对实验中建模的所有模块进行功能描述，并列出各模块建模的 verilog 代码）

```
module Synchronous_D_FF(  
input CLK, //时钟信号，上升沿有效  
input D, //输入信号 D  
input RST_n, //复位信号，低电平有效  
output reg Q1, //输出信号 Q  
output reg Q2//输出信号?  
);  
always @ (posedge CLK)  
begin  
    if(RST_n && D)  
    begin  
        Q1<=1;  
        Q2<=0;  
    end  
    else
```

```

begin
    Q1<=0;
    Q2<=1;
end
end
endmodule

```

四、测试模块建模

（要求列写各建模模块的 test bench 模块代码）

```

module Synchronous_D_FF_tb();
    reg CLK, D, RST_n;
    wire Q1, Q2;

    Synchronous_D_FF uut(.CLK(CLK), .D(D), .RST_n(RST_n), .Q1(Q1), .Q2(Q2));

    initial
        CLK = 0;
    always
        #10 CLK = ~CLK;

    initial
    begin
        D=1;
        RST_n=0;
        #50
        D=1;
        RST_n=1;
        #50
        D=0;
        RST_n=0;
        #50
        D=0;
        RST_n=1;
    end

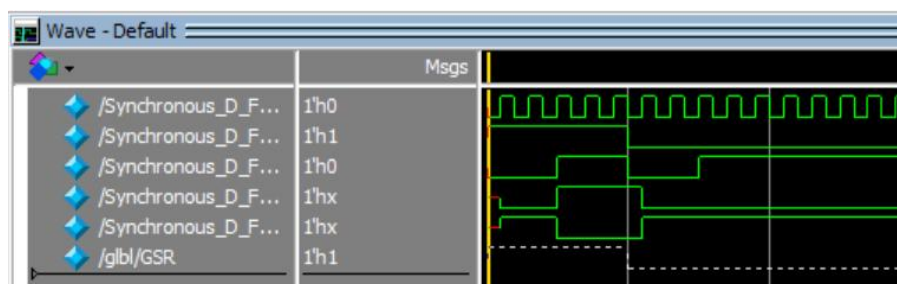
endmodule

```

五、实验结果

（该部分可截图说明，要求 logisim 逻辑验证图、modelsim 仿真波形图、以及下板后的实验结果贴图（实验步骤中没有下板要求的实验，不需要下板贴图））

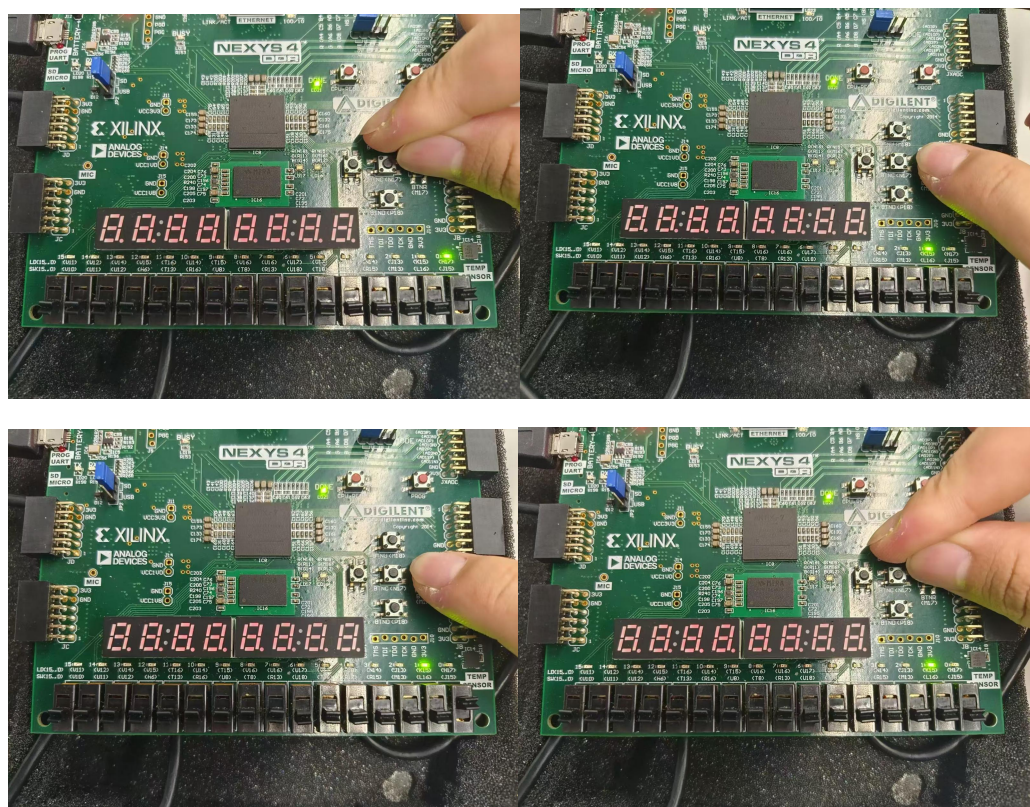
Modelsim 仿真波形图：



综合下板

D 触发器

变量	CLK	D	RST_n	Q1	Q2
N4 板上的管脚	BTNR (M17)	SW0 (J15)	BTNU (M18)	LD1 (H17)	LD2 (K15)



经验证，符合逻辑。

6.6_2 异步复位 D 触发器

一、实验内容

b) D 触发器

D 触发器是在 SR 触发器基础上构建的。即在一个 SR 触发器上增加一个非门。图 6.6.2

给出了一个 D 触发器的逻辑图及其功能表。该触发器只有一个数据输入端 D，经反相器反相后，变成互补数据输入，送到 SR 触发器，从而避免了 SR 触发器存在的不稳态问题。当数据输入 D=1，且在时钟脉冲上升沿，触发器置位（1 状态）；当数据输入 D=0，且在时钟脉冲上升沿，触发器复位（0 状态）。



图 6.6.2 D 触发器逻辑图及其功能表

所谓的异步复位，即无论时钟边沿到来与否，只要复位信号有效输出就会被复位。

二、硬件逻辑图：无

三、 模块建模

```
module Asynchronous_D_FF(  
input CLK, //时钟信号，上升沿有效  
input D, //输入信号 D  
input RST_n, //复位信号，低电平有效  
output reg Q1, //输出信号 Q  
output reg Q2//输出信号??  
);  
  
always @ (posedge CLK or negedge RST_n)  
begin  
    if (!RST_n || !D)  
        begin  
            Q1 <= 0;  
            Q2 <= 1;  
        end  
    else  
        begin  
            Q1 <= 1;  
        end  
    end
```

```
        Q2 <= 0;
    end
end
endmodule
```

四、测试模块建模

```
module Asynchronous_D_FF_tb();
    reg CLK, D, RST_n;
    wire Q1, Q2;

    Asynchronous_D_FF uul(CLK,D,RST_n, Q1,Q2);

    initial
        CLK=0;
    always
        #40 CLK=~CLK;

    initial
    begin
        D=1;
        RST_n=1;
        #60

        RST_n=0;
        #10

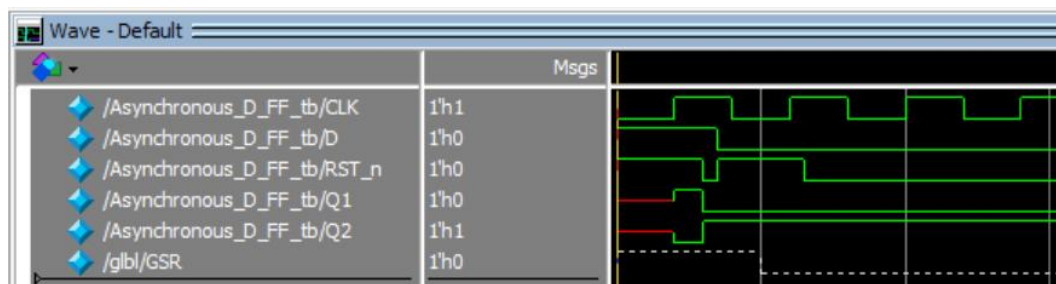
        D=0;
        RST_n=1;
        #60;

        D=0;
        RST_n=0;
    end

endmodule
```

五、实验结果

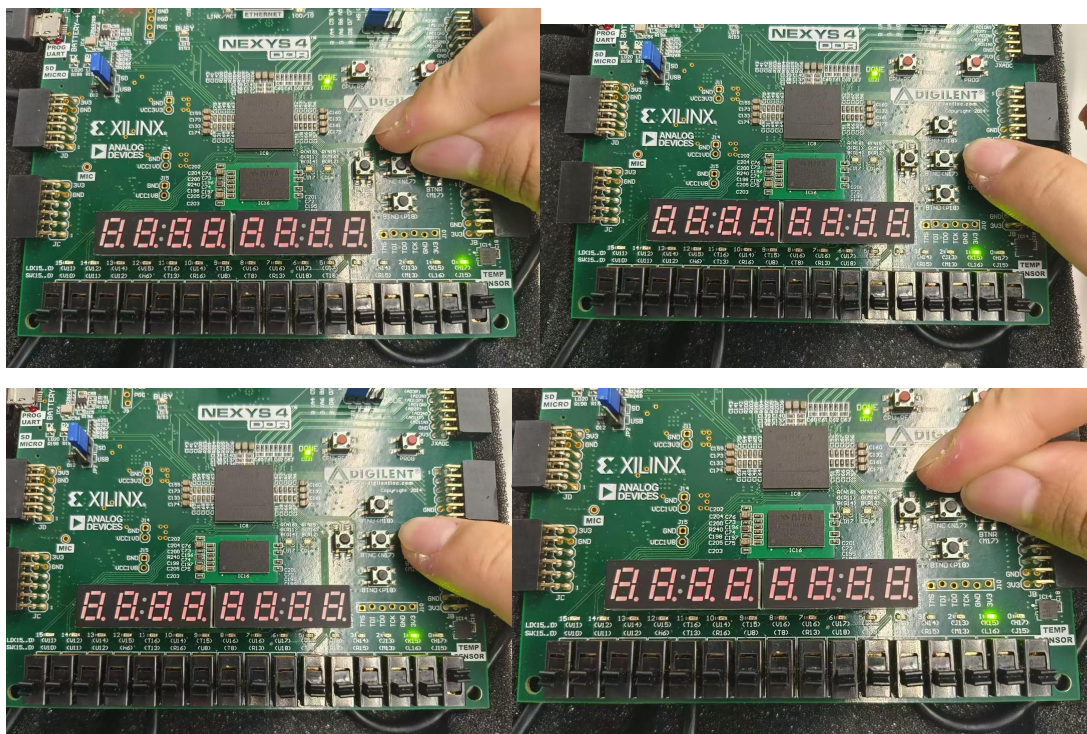
1.Modelsim 仿真波形图



2.综合下板

D 触发器

变量	CLK	D	RST_n	Q1	Q2
N4 板上的管脚	BTNR (M17)	SW0 (J15)	BTNU (M18)	LD1 (H17)	LD2 (K15)



经验证，符合逻辑。

6.6_3 异步复位 JK 触发器

一、实验内容

c) JK 触发器

JK 触发器是一种广泛应用的触发器类型。字母 J 和 K 只表示它们是两个数据输入端符号，没有什么特别含义。JK 触发器与 SR 触发器在置位、复位方面的功能是相同的，不同之处在于，JK 触发器改进了 SR 触发器存在的不稳定状态。当 $J=1$ ， $K=1$ 时，对每一个连续的时钟脉冲，触发器可改变成相反状态或计数状态，这种工作方式称为交替操作。图 6.6.3 所示为 JK 触发器的内部逻辑及其功能表。

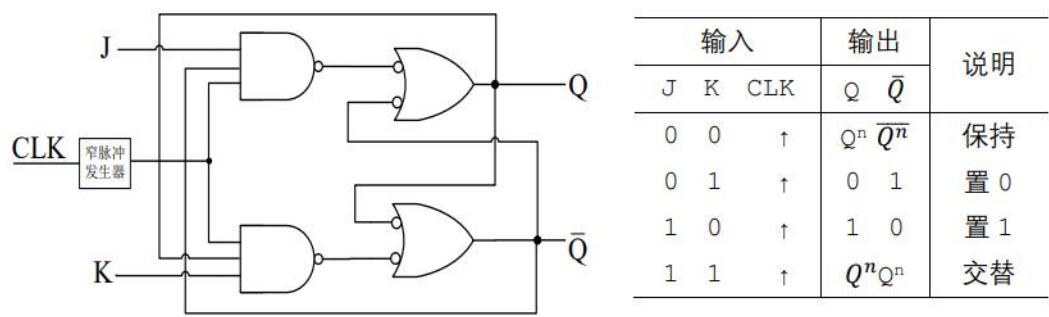


图 6.6.3 JK 触发器逻辑图及其功能表

二、硬件逻辑图：无

三、模块建模

```
module JK_FF(  
input CLK, //时钟信号，上升沿有效  
input J, //输入信号 J  
input K, //输入信号 K  
input RST_n, //复位信号，低电平有效  
output reg Q1, //输出信号 Q  
output reg Q2 //输出信号 Q?  
);  
  
always @ (posedge CLK or negedge RST_n)  
begin  
    if(!RST_n) begin  
        Q1=0;  
        Q2=1;  
    end  
    else begin  
        if(J&&K) begin
```

```

        Q1=!Q1;
        Q2=!Q2;
    end
    else if(J&&!K) begin
        Q1=1;
        Q2=0;
    end
    else if(!J&&K) begin
        Q1=0;
        Q2=1;
    end
    else;
end
end
endmodule

```

四、测试模块建模

```

module JK_FF_tb();
    reg CLK, J, K, RST_n;
    wire Q1,Q2;

    JK_FF uut(CLK, J, K, RST_n, Q1, Q2);

    initial
        CLK=0;
    always
        #10 CLK=~CLK;

    initial begin
        RST_n = 1; J = 1;K = 0;
        #40 J = 1; K = 1;
        #40 J = 0; K = 0;
        #40 J = 0; K = 1;
        #40 J = 1; K = 0;

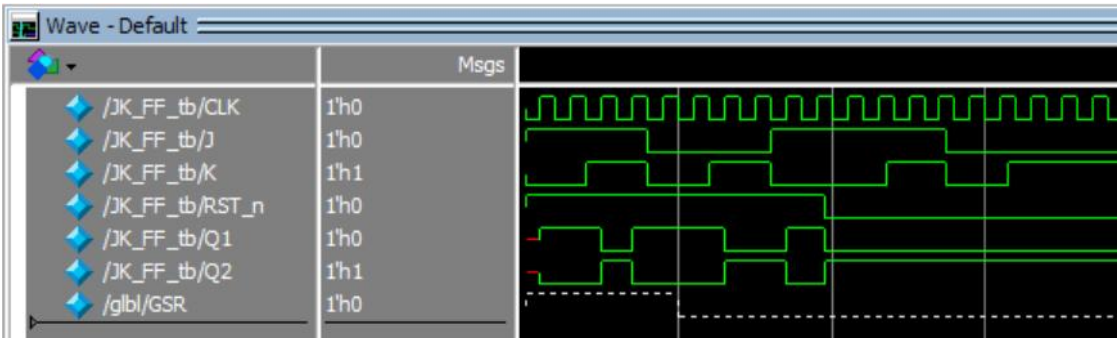
        #35 RST_n = 0; J = 1; K = 0;
        #40 J = 1; K = 1;
        #40 J = 0; K = 0;
        #40 J = 0; K = 1;

    end
endmodule

```

五、实验结果

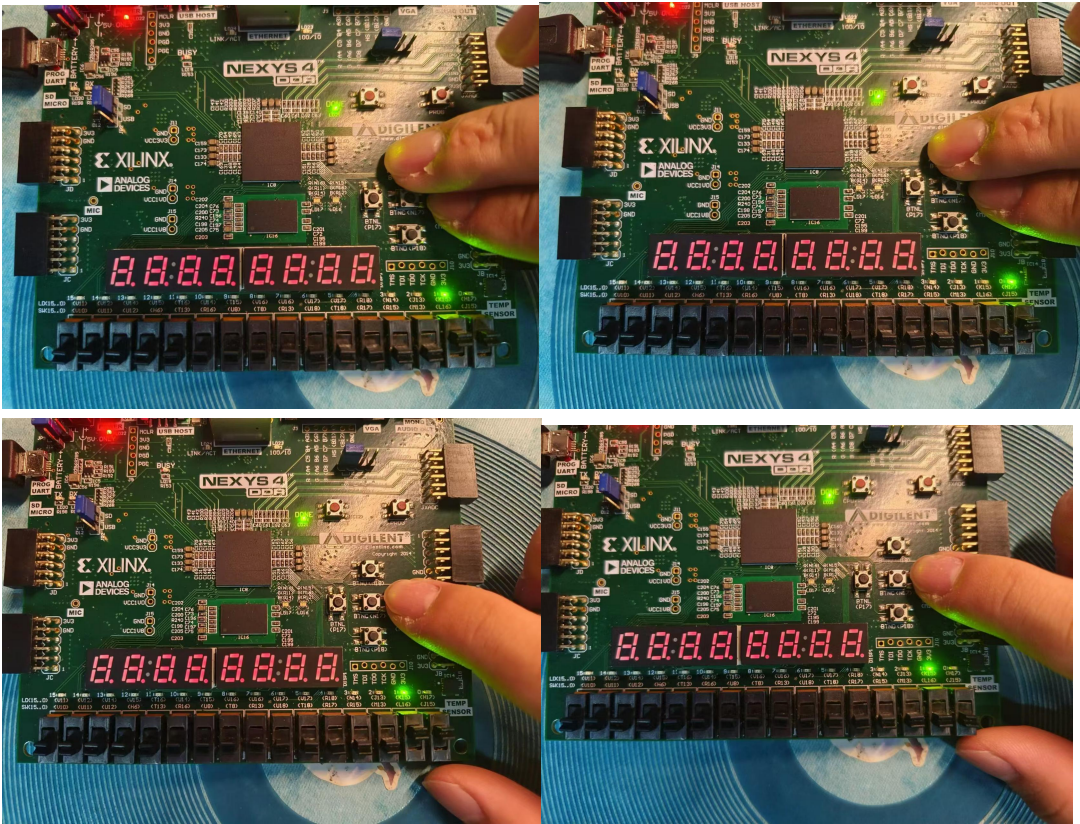
Modelsim 仿真波形图

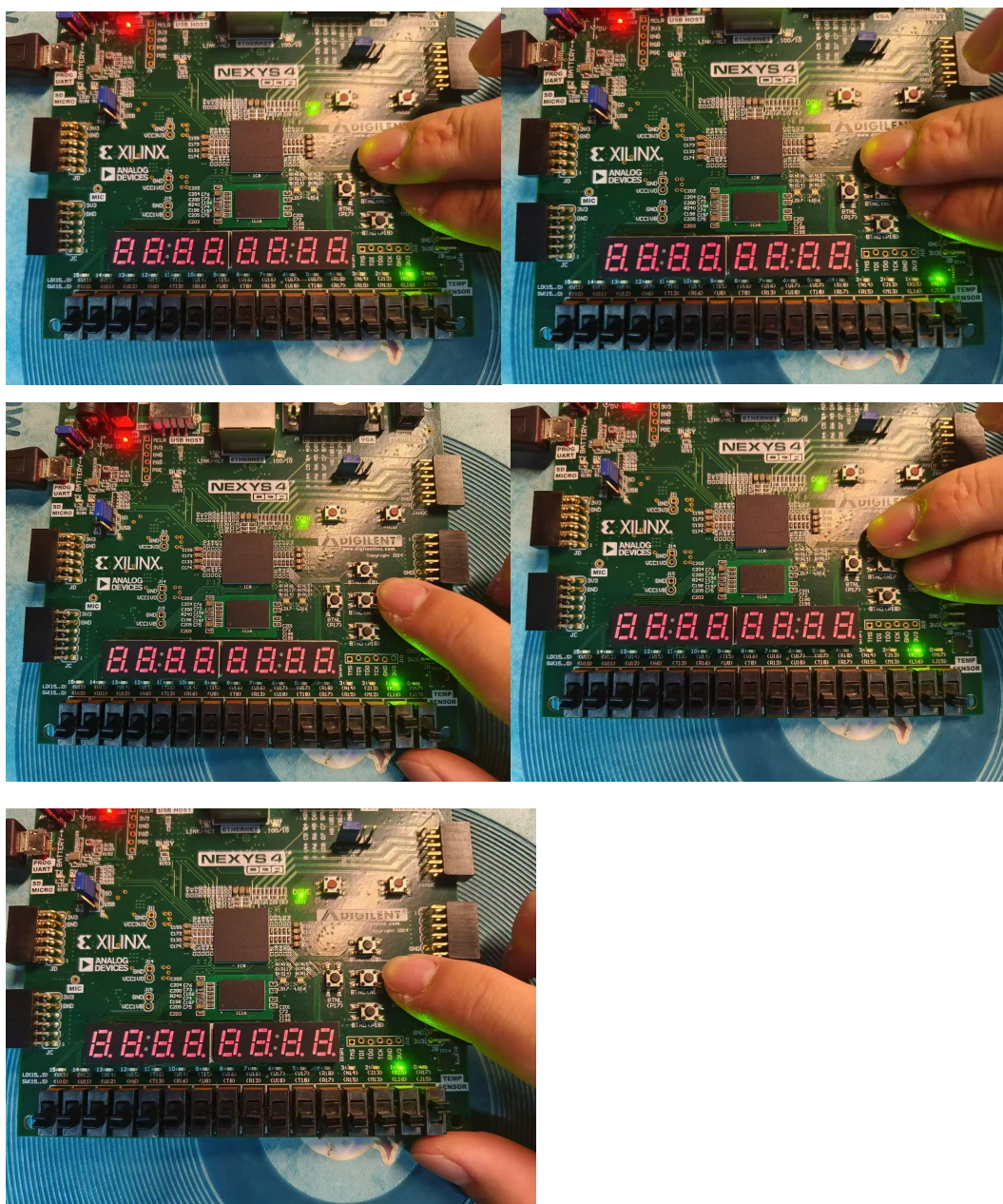


综合下板

JK 触发器

变量	CLK	J	K	RST_n	Q1	Q2
N4 板上的管脚	BTNR (M17)	SW0 (J15)	SW1 (L16)	BTNU (M18)	LD1 (H17)	LD2 (K15)





经验证，符合逻辑。

6.6_4 PC 寄存器

一、实验内容

2) PC 寄存器

PC 寄存器 (program counter) 是组成 CPU 的基本部件, 用来存放当前正在执行的指令, 包括指令的操作码和地址信息。我们知道, D 触发器可以用于存储比特信号。如果 D 为 1, 那么在时钟的上升沿, D 触发器的输出 Q 将变为 1。如果 D 为 0, 那么在时钟的上升沿, D 触发器的输出 Q 将变为 0。在实时数字系统中, 通常 D 触发器的时钟输入端始终有时钟信号输入。这就意味着在每个时钟的上升沿, 当前 D 值都将被锁存在 Q 中。在本实验主

利用 Verilog HDL 模块实例化基于 D 触发器模块实现一个 PC 寄存器。图 6.6.4 给出了所要建模的 PC 寄存器其功能图。

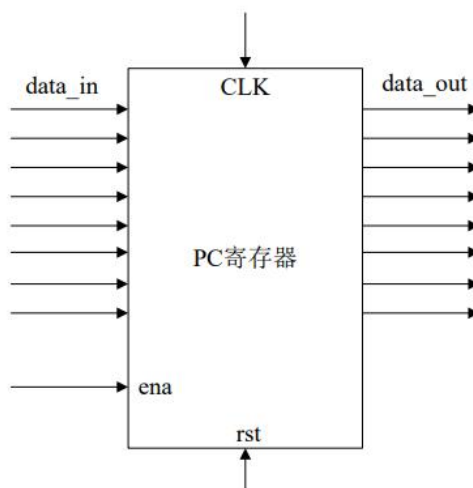


图 6.6.4 PC 寄存器功能图

二、硬件逻辑图：无

三、模块建模

```
module pcreg(  
input clk, //1 位输入, 寄存器时钟信号, 上升沿时为 PC 寄存器赋值  
input rst, //1 位输入, 异步重置信号, 高电平时将 PC 寄存器清零  
//注: 当 ena 信号无效时, rst 也可以重置寄存器  
input ena, //1 位输入, 有效信号高电平时 PC 寄存器读入 data_in 的值, 否则保持原有输出  
input [31:0] data_in, //32 位输入, 输入数据将被存入寄存器内部  
output reg [31:0] data_out //32 位输出, 工作时始终输出 PC 寄存器内部存储的值  
  
always @(posedge clk, posedge rst)
```

```

begin
    if(rst) begin
        data_out[31:0]<=32'b0000_0000_0000_0000_0000_0000_0000;
    end
    else begin
        if(ena)
            data_out<=data_in;
    end
end

endmodule

```

四、测试模块建模

```

module pcreg_tb();
    reg clk;
    reg rst;
    reg ena;
    reg [31:0] data_in;
    wire [31:0] data_out;

    pcreg uut(clk, rst, ena, data_in, data_out);

    initial begin
        clk=0;
        rst=0;
        ena=1;
        data_in=32'b0000_0000_0000_0000_0000_0000_0000_0000;
    end

    always
        #20 clk=~clk;

    initial begin
        #40 data_in=32'b0000_0000_1111_1111_0000_0000_0000_0000;
        #40 data_in=32'b1111_1111_0000_1111_0000_0000_1111_1111;

        #40 ena=0;
        #40 data_in=32'b0000_0000_1111_1111_0000_0000_0000_0000;
        #40 ena=1;

        #10 rst=1;
        #40 data_in=32'b1111_1111_0000_1111_0000_0000_1111_1111;
    end
endmodule

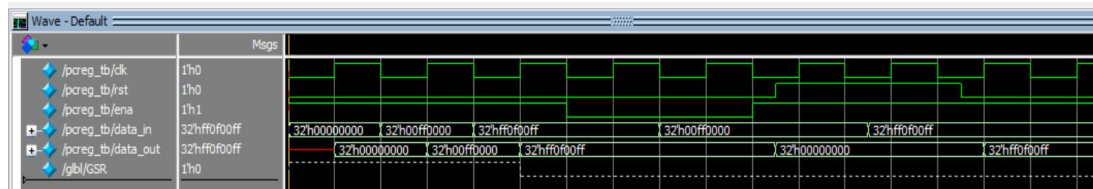
```

```
        #40 rst=0;
    end

endmodule
```

五、实验结果

Modelsim 仿真波形图



（本实验无需下板）
经验证，符合逻辑。